Análise de Dados com Python [T6]

Situação de Aprendizagem (Projeto Avaliativo) - Módulo 1 - Semana 13

SUMÁRIO

1 CONTEXTUALIZAÇÃO 1
2 DESAFIO 1
3 RESULTADOS ESPERADOS (ENTREGA) 2
4 REQUISITOS DA APLICAÇÃO 2
5 ROTEIRO DA APLICAÇÃO 3
5.1 FORMATO DO PIPELINE 3
5.2 ETAPAS DO PIPELINE 3
5.3 ARQUITETURA PROPOSTA 4
5.4 DICIONÁRIO DE DADOS DA CAMADA SILVER 4
5.5 CONSTRAINTS 7
5.6 CONVERSÕES DE TRATAMENTO DE DADOS 7
5.7 DOCUMENTAÇÃO NO README.MD 7
5.8 USO DO GITHUB 7
6 CRITÉRIOS DE AVALIAÇÃO 8

1 CONTEXTUALIZAÇÃO

Atualmente, a transparência pública permite a geração de grandes volumes de dados abertos. Um deles refere-se à base de Viagens a Serviço do Portal da Transparência do Governo Federal. Entretanto, estes dados são disponibilizados em sua forma bruta e, transformá-los em informação confiável é trabalho do Engenheiro de Dados, que, por sua vez, extrai, limpa, modela e analisa. Dominar um pipeline de dados (ETL) e a Arquitetura Medallion (camadas Raw, Silver e Gold) com Python e SQL é uma das competências mais valorizadas na área de dados atualmente, uma vez que permite uma utilização eficiente e segura.

2 DESAFIO

Você participa de uma consultoria de dados que foi contratada pelo governo para dar mais transparência aos gastos públicos com viagens a serviço. O órgão responsável já publica os dados no Portal da Transparência, mas eles chegam brutos,

desorganizados e a equipe precisa com urgência avaliar como anda o fluxo das informações das viagens.

A sua missão é construir, do zero, um pipeline de dados que baixe essas informações, preservando o histórico original, limpar a estrutura e transformar os dados brutos em métricas e gráficos claros.

3 RESULTADOS ESPERADOS (ENTREGA)

Com o desafio da construção do pipeline de dados de ponta a ponta, você irá transformar dados brutos e desorganizados do Portal da Transparência em informação confiável para a tomada de decisão.

Ao final do projeto, você terá construído um pipeline completo que:

- Baixa os dados de Viagens a Serviço direto da fonte oficial, sem intervenção manual;
- Preserva o dado original fielmente na camada Raw, garantindo rastreabilidade e auditoria de qualquer transformação futura;
- Limpa e organiza os dados na camada Silver, com tipagem correta e integridade referencial entre as tabelas , eliminando inconsistências, duplicidades e erros de formato;
- Responder perguntas de negócio reais na camada Gold, tudo isso apoiado por métricas e gráficos que tornam a informação acessível para quem for tomar decisões.

As perguntas de negócio a serem respondidas serão:

- Os 5 órgãos com maior custo total?
- Os 3 destinos com maior custo médio por viagem?
- A viagem de maior duração e seu custo total?
- Qual o tipo de pagamento com maior valor médio?
- Qual o meio de transporte mais usado nos trechos?
- Qual UF de destino aparece em mais trechos?
- Qual órgão pagou mais no total ?

Não serão aceitos projetos submetidos após a data limite da atividade, e, ou alterados depois de entregues.

Importante:

1. Será considerado como data final de entrega a última atualização no repositório do projeto no GitHub. Lembre-se de não modificar o código até receber sua nota.
2. Não esqueça de submeter o link no AVA. O link não submetido terá penalidade na nota.
3. Deverão ser disponibilizados os links na tarefa **Módulo 1 - Projeto Avaliativo**, presente na semana 13 do AVA até o dia **03/11/2026 às 22h.**

4 REQUISITOS DA APLICAÇÃO

Abaixo seguem os requisitos a serem desenvolvidos, organizados pelas fases do pipeline (Arquitetura Medallion). O dicionário de dados completo da camada Silver está nos documentos colocados 👉 **no drive**.

- **Fase 0 - Banco e tabelas (0_criar_banco.sql):** criar o database e as 8 tabelas. As 4 tabelas Raw têm todas as colunas VARCHAR e sem constraints; as 4 tabelas Silver são tipadas e têm PRIMARY KEY, FOREIGN KEY e mais 2 constraints por tabela (NOT NULL, CHECK e UNIQUE), declaradas dentro do CREATE TABLE.

- **Fase 1 - Extração e camada Raw (1_extrair.py):** baixar 👉 **arquivo .zip do Google Drive**, ler os 4 CSVs em blocos e carregar nas tabelas Raw sem alterar o conteúdo. O processo deve ser idempotente (TRUNCATE antes de carregar) e resiliente (try/except). **Este zip já está transformado para trabalharmos com 6 meses do ano 2025.**

- **Fase 2 - Transformação e camada Silver (2_transformar.py):** copiar Raw para Silver convertendo os tipos (texto para DECIMAL e DATE), respeitando a integridade referencial e calculando as colunas valor_total e duracao_dias.

- **Fase 3 - Camada Gold (3_analise.ipynb):** responder 3 perguntas de negócio com consulta SQL, tabela e gráfico, e construir uma camada Gold agregada com JOIN + GROUP BY (criada no MySQL/PostgreSQL como tabela e como VIEW), com as outras 4 perguntas de negócio e 3 gráficos.

  Boas práticas: credenciais apenas no arquivo .env (fora do código), .gitignore adequado e código organizado (PEP-8 e modularização).

5 ROTEIRO DA APLICAÇÃO

Abaixo segue um detalhamento de como o pipeline deve ser desenvolvido e estruturado.

5.1 FORMATO DO PIPELINE

O projeto é um pipeline de dados em Python + MySQL/PostgreSQL, com os arquivos numerados pela ordem de execução:

0_criar_banco.sql,
1_extrair.py,
2_transformar.py
3_analise.ipynb, seguindo a Arquitetura Medallion (Raw, Silver e Gold).

As credenciais ficam no arquivo .env.

5.2 ETAPAS DO PIPELINE

ARQUIVO	ESTADO	DESCRIÇÃO
config.py	pronto	Parâmetros + leitura do .env. Falta só você colar o DRIVE_FILE_ID.
banco.py	pronto	Conexão e funções utilitárias do MySQL/PostgreSQL (conectar, executar, inserir_em_lote).
.env.example	pronto	Modelo de credenciais (copie para .env).
.gitignore	entregar	Já ignora .env, .zip, .csv, data/.
requirements.txt	entregar	Dependências do projeto.
0_criar_banco.sql	entregar	Criar as 8 tabelas (4 Raw + 4 Silver) com PK, FK e constraints.
1_extrair.py	entregar	Download + leitura dos CSVs + carga na camada Raw.
2_transformar.py	entregar	Limpeza e tipagem (Raw → Silver) + colunas calculadas.
3_analise.ipynb	entregar	Camada Gold: perguntas de negócio + gráficos + Gold agregada.

5.3 ARQUITETURA PROPOSTA

CAMADA	O QUE É	ONDE VAI FICAR
Raw	Cópia fiel do CSV. Todas as colunas VARCHAR (mantém vírgulas, datas DD/MM/AAAA, formatos originais).	raw_viagem, raw_pagamento, raw_passagem, raw_trecho
Silver	Dados limpos e tipados (DECIMAL, DATE), com PK e FK declaradas.	silver_viagem, silver_pagamento, silver_passagem, silver_trecho
Gold	Métricas de negócio, agregações e gráficos.	3_analise.ipynb (+ tabelas gold_*)

Fonte: os 4 CSVs dentro do .zip são 2025_Viagem.csv, 2025_Pagamento.csv, 2025_Passagem.csv e 2025_Trecho.csv. Eles usam separador **;** , encoding latin-1, vírgula como decimal (ex.: 1272,97) e datas DD/MM/AAAA.

5.4 DICIONÁRIO DE DADOS DA CAMADA SILVER

TABELA silver_viagem

COLUNA	TIPO	RESTRIÇÕES
id_viagem	VARCHAR(20)	PRIMARY KEY, NOT NULL
num_proposta	VARCHAR(20)	
situacao	VARCHAR(50)	
viagem_urgente	VARCHAR(5)	
cod_orgao_superior	VARCHAR(20)	
nome_orgao_superior	VARCHAR(255)	NOT NULL (constraint extra)
nome_viajante	VARCHAR(255)	
cargo	VARCHAR(255)	
data_inicio	DATE	convertido de DD/MM/AAAA

data_fim	DATE	convertido de DD/MM/AAAA
destinos	VARCHAR(4000)	
motivo	VARCHAR(4000)	
valor_diarias	DECIMAL(10,2)	CHECK >= 0 (constraint extra)
valor_passagens	DECIMAL(10,2)	
valor_devolucao	DECIMAL(10,2)	
valor_outros_gastos	DECIMAL(10,2)	
valor_total	DECIMAL(12,2)	calculado
duracao_dias	INT	calculado

TABELA silver_passagem

COLUNA	TIPO	RESTRIÇÕES
id_passagem	INT	PRIMARY KEY, AUTO_INCREMENT
id_viagem	VARCHAR(20)	FOREIGN KEY → silver_viagem(id_viagem), NOT NULL
meio_transporte	VARCHAR(50)	
pais_origem_ida	VARCHAR(60)	
uf_origem_ida	VARCHAR(40)	
cidade_origem_ida	VARCHAR(80)	
pais_destino_ida	VARCHAR(60)	
uf_destino_ida	VARCHAR(40)	
cidade_destino_ida	VARCHAR(80)	
valor_passagem	DECIMAL(10,2)	CHECK >= 0 (constraint extra)
taxa_servico	DECIMAL(10,2)	CHECK >= 0 (constraint extra)
data_emissao	DATE	convertido de DD/MM/AAAA

TABELA silver_pagamento

COLUNA	TIPO	RESTRIÇÕES

id_pagamento	INT	PRIMARY KEY, AUTO_INCREMENT
id_viagem	VARCHAR(20)	FOREIGN KEY → silver_viagem(id_viagem), NOT NULL
num_proposta	VARCHAR(20)	
nome_orgao_pagador	VARCHAR(255)	
nome_ug_pagadora	VARCHAR(255)	
tipo_pagamento	VARCHAR(50)	NOT NULL (constraint extra)
valor	DECIMAL(10,2)	CHECK >= 0 (constraint extra)

TABELA silver_trecho

COLUNA	TIPO	RESTRIÇÕES
id_trecho	INT	PRIMARY KEY, AUTO_INCREMENT
id_viagem	VARCHAR(20)	FOREIGN KEY → silver_viagem(id_viagem), NOT NULL
sequencia_trecho	INT	
origem_data	DATE	
origem_uf	VARCHAR(40)	
origem_cidade	VARCHAR(80)	
destino_data	DATE	
destino_uf	VARCHAR(40)	
destino_cidade	VARCHAR(80)	
meio_transporte	VARCHAR(50)	
numero_diarias	DECIMAL(10,2)	CHECK >= 0 (constraint extra)

As tabelas Raw têm as mesmas colunas, porém todas VARCHAR e sem constraints. A Raw inclui colunas a mais que a Silver não usa (ex.: cpf_viajante, funcao, dados de volta da passagem) , na Raw, replique o CSV inteiro.

5.5 CONSTRAINTS

Declare, dentro do CREATE TABLE de cada tabela Silver, 2 constraints (8 no total), usando NOT NULL, CHECK e UNIQUE:

TABELA	CONSTRAINT 1	CONTRAINT 2
silver_viagem	NOT NULL em nome_orgao_superior	CHECK em valor_diarias >= 0
silver_pagamento	CHECK em valor >= 0	NOT NULL em tipo_pagamento
silver_passagem	CHECK em valor_passagem >= 0	CHECK em taxa_servico >= 0
silver_trecho	CHECK em numero_diarias >= 0	UNIQUE em (id_viagem, sequencia_trecho)

5.6 CONVERSÕES DE TRATAMENTO DE DADOS

Aqui você terá que desenvolver o tratamento com funções específicas para os campos de valores e taxas(texto para decimal) e datas (texto para data).

5.7 DOCUMENTAÇÃO NO README.MD

Crie um arquivo readme.md no repositório do seu projeto no GitHub, para documentar a sua solução, bem como demonstrar as técnicas e linguagens utilizadas, além do escopo do projeto e como o usuário pode executar o seu sistema.

Algumas dicas interessantes para utilizar na criação do seu portfólio são:

- Descrever qual o problema ele resolve;
- Descrever quais técnicas e tecnologias utilizadas. Aqui você também pode inserir alguma imagem ou diagrama para melhor entendimento;
- Descrever como executar;
- Descrever quais melhorias podem ser aplicadas;
- Descrever as conclusões e os insights a partir dos gráficos e análise de base.

5.8 USO DO GITHUB

Utilizar um repositório GitHub para controle de versionamento do projeto. Siga as diretrizes de boas práticas de uso do Git abaixo:

- Seguir *commits* com descrição sucinta e direta e, de preferência, com poucos arquivos alterados por *commit*.

- Prefira mensagens como “implementa X” ou “corrige Y”, em vez de “X foi implementado” ou “Y foi corrigido”.
- Crie pastas para o versionamento do projeto, conforme explicado nas aulas.

6 CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto.

Ele possui variação de nota de 0 (zero) a 10 (dez) como nota mínima e máxima.
Serão desconsiderados e atribuída a nota 0 (zero) os projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas.

Lembre-se: Você está livre para utilizar outras soluções como base, mas não é permitida a cópia.

Uso do GitHub e Readme.md				
N°	Critério de Avaliação	0	0,75	1,5
1	Versionamento com branches e commits	O repositório do projeto não apresenta branches e commits.	O repositório do projeto apresenta parte das branches e commits distintos e nomeadas padronizadamente para cada funcionalidade desenvolvida.	O repositório do projeto apresenta branches e commits distintos e nomeadas padronizadamente para cada funcionalidade desenvolvida.

N°	Critério de Avaliação	0	0,5	1,0
2	Organização dos arquivos no repositório	O repositório do projeto não apresenta os arquivos (.env, .sql, .csv, .py e README.md) estruturados conforme as instruções.	O repositório do projeto apresenta parte dos arquivos (.env, .sql, .csv, .py e README.md) estruturados conforme as instruções.	O repositório do projeto apresenta os arquivos (.env, .sql, .csv, .py e README.md) estruturados conforme as instruções.

Nº	Critério de Avaliação	0	0,50
3	**Extração e criação da camada Raw**	Não fez extração ou não existe camada Raw.	Realizou a extração automatizada (download + carga), com tratamento de exceções (try/except) e sem duplicar registros em reexecuções.

Nº	Critério de Avaliação	0	0,5	1,0
4	**Tipagem e limpeza**	Os dados permanecem como texto.	A conversão foi feita apenas em partes dos campos, ou com inconsistências.	Todos os campos estão corretamente convertidos, sem inconsistências.
5	**Camada Gold com funções de agregação e JOINS**	A camada Gold não foi implementada.	Usou apenas JOINS ou GROUP BY, ou há erro de lógica empregada	Apresentou a criação de uma tabela agregada por meio da junção de tabelas (JOIN) e agrupamento de dados (GROUP BY), refletindo as regras de negócio estipuladas.

Nº	Critério de Avaliação	0	0,5	1,0	1,5
6	**Perguntas de negócio**	Não há pergunta formulada ou respondida.	As perguntas foram formuladas, mas sem respostas ou com respostas incorretas	As perguntas respondidas, mas com pouca profundidade analítica.	As perguntas de negócio foram pertinentes e respondidas com análises baseadas em evidências extraídas do tratamento e visualização do conjunto de dados fornecido.
7	**Gráficos com dataviz aplicado**	Não foi apresentado gráfico.	Os gráficos não possuem eixos nomeados ou são de difícil leitura	Os Eixos estão nomeados, mas o tipo de gráficos é inadequado ou faltando elementos básicos como título/legendas	As visualizações gráficas apresentam a identificação completa de seus elementos estruturais (título, eixos nomeados e legenda), refletindo os dados extraídos do projeto.

Nº	Critério de Avaliação	0	0,5	1,0	1,5	2,0

8	**Modelagem e camada Silver**	Não há modelagem	Os dados na camada Silver não possuem modelagem relacional.	Definiu a PK, mas sem as FK nem constraints	Definiu a PK e a FK, mas as constraints estão incompletas ou inconsistentes.	O script de criação da tabela apresenta a definição explícita de chaves primárias (PK), chaves estrangeiras (FK) e restrições (constraints), assegurando a integridade referencial entre as tabelas.